EEL 5764: Computer Architecture

Lectures 1–2: Course Introduction & Fundamental Overview

Sandip Ray
Warren B. Nelms Endowed Professor
Department of Electrical & Computer Engineering
University of Florida

Required Reading: H&P Chapter 1, P&H Chapter 1

About Your Instructor: Dr. Sandip Ray

•Professional Profile & Coordinates

•Position: Warren B. Nelms Endowed Professor, Electrical & Computer Engineering Dept, UF.

•Contact: sandip@ece.ufl.edu | Office: MALA 4131.

•Industry Experience: Bridging commercial microarchitectural development and advanced research labs.

•Intel Labs: Spent years working on microarchitecture design, performance modeling, and post-silicon validation frameworks.

•NXP Semiconductors: Headed safety-critical automotive computing architectures and hardware-level validation protocols.

•Research Interests & Course Synergy

•Secure, Intelligent & Resilient Architectures: Focus on designing hardware that is robust against both physical and cyber vectors.

•System-on-Chip (SoC) Security: Researching hardware-level enclaves and protections against microarchitectural side-channels.

•Robust Autonomous Vehicles: Architecting computing platforms that operate deterministically under harsh environmental conditions.

•Trustworthy Machine Learning: Designing specialized AI hardware accelerators (NPUs) that guarantee safety and privacy.

•Course Connection: You aren't just learning textbook principles. We will frequently bridge classic models with modern research-level design challenges.

Course Philosophy & Key Learning Objectives

Focus 1: A Quantitative Approach to Design

Moving beyond intuition: Computer architecture today is an empirical science governed by measurement.

Evaluating design alternatives requires quantitative metrics (e.g., execution time, energy, yield, cost).

We will emphasize the quantitative evaluation of tradeoffs rather than purely qualitative features.

Learning to find optimal design choices under severe hardware constraints (power, complexity, area).

Focus 2: The Dynamic Hardware/Software Boundary

The compiler, operating system, and instruction set architecture (ISA) must work in unified harmony.

Modern systems require cross-layer optimization to achieve security and performance.

We will study how software constructs (compilers, loops, threads) exploit complex hardware structures.

Developing a systems perspective: understanding how microarchitectural choices affect compiler design.

Prerequisites & Expected Technical Background

To succeed in this course, you should have a solid foundation in both hardware principles and systems programming. We leverage both throughout the semester.

Hardware Requirements (Digital Design)

- Understanding of basic combinational and sequential logic design.
- Familiarity with registers, state machines, and clocking.
- Basic processor organization concepts (registers, ALU, instruction fetching).

Software & Systems Requirements

- Strong C/C++ programming proficiency (pointers, memory model).
- Comfort in a Linux command-line environment (crucial for tool runs).
- Basic assembly language concepts.

💡 **Background Consultation: Please talk to the instructor early if you are unsure about your background. We are here to support you.**

System Tooling: Why gem5 architectural simulator?

Computer architecture research and design rely heavily on detailed simulation models rather than hardware prototyping.

Industry Standard

gem5 is a highly popular, modular, and detailed architectural simulator used extensively in academia and major companies (e.g., ARM, AMD, Google) to evaluate future hardware architectures before they are fabricated.

Full-System Accuracy

It supports cycle-level modeling, detailed cache and memory hierarchies, virtual memory, and diverse instruction sets (ARM, x86, RISC-V), allowing us to boot real operating systems on simulated CPUs.

Hands-on Homeworks

You will use gem5 to conduct quantitative experiments, modify cache sizes, analyze pipelining bottlenecks, and observe processor performance curves in action across our four homework assignments.

⚠ **gem5 Simulation Setup: Because compiling and running gem5 simulation models can be computationally heavy and resource-intensive, Homework 1 may include a basic readiness exercise to ensure everyone is set up early.**

Textbooks & Reading Materials

Required Primary Textbook

Computer Architecture: A Quantitative Approach

Authors: John L. Hennessy and David A. Patterson

Publisher: Morgan Kaufmann

The absolute standard reference for graduate-level computer systems research.

Essential reading for instruction-level parallelism, advanced caches, and multicore systems.

Assigned weekly readings will form the foundational material for lecture topics.

Additional Recommended Material

Computer Organization and Design RISC-V Edition: The Hardware/Software Interface

Authors: David A. Patterson and John L. Hennessy

Publisher: Morgan Kaufmann

Provides an excellent refresher on core processor microarchitectures.

Specifically useful for understanding our RISC-V ISA lectures and homework implementations.

Perfect bridging resource for students needing a baseline conceptual boost.

Evaluation and Grading Policy

Participation

5%

Engagement in class discussions & active contributions to Canvas.

Homeworks

30%

4 assignments (hands-on gem5 + analysis). Lowest score is dropped!

Midterm Exam

25%

Thursday, October 22, 2026. Midterm expected to use Honorlock (subject to confirmation). No final exam!

Semester Project

40%

Significant collaborative research, simulation, or implementation.

💡 **absolute Grading Scale applies (no curve) | Active class participation can bump borderline grades up to 1.0% higher.**

Homework Assignments & Policies

The course features 4 comprehensive homework assignments that merge rigorous mathematical analysis of architecture with practical simulator runs.

Homework Timeline

Homework 1: Assigned Sept 1 | Due Sept 10 (gem5 Environment Setup)

Homework 2: Assigned Sept 24 | Due Oct 6 (Pipeline Timing & Hazards)

Homework 3: Assigned Oct 29 | Due Nov 10 (Cache Hierarchy & Main Memory)

Homework 4: Assigned Nov 12 | Due Dec 1 (Vector & Multicore Parallelism)

The Drop-Lowest Policy

Because graduate-level simulations in gem5 can introduce technical roadblocks and varying run-times, we establish a strict safety net:

Your lowest homework score of the four is dropped automatically.

Late Policy: Up to 3 late days are permitted for homework submissions, subject to a penalty (e.g., 10% deduction per day), but no assignments are accepted after this 3-day grace window.

Homework assignments represent 30% of your total grade. Do not miss them.

The Semester Project (40% Weight)

The semester project is a highly significant research-oriented component designed for team collaboration.

Project Release

Sept 17

Guidelines, tool requirements, and a curated list of research topics are published on Canvas.

Proposal (15%)

Oct 1

Teams submit a 2-page detailed study outline, problem definition, and planned gem5 setups.

Video & Peer Discussion (50%)

Nov 17 - Nov 20

Nov 17: 5-Min Video Presentation Due (30%)

- Nov 19: Assigned Peer Comments & Questions Due
- Nov 20: Team Responses Due / Canvas Discussion Closes (20%)

Written Report (35%)

Dec 1

Final high-quality, IEEE-format technical paper summarizing methodology, plots, and code results due on Canvas by midnight.

Project Topic Workshop: September 22, 2026. A dedicated session to form teams and scope out initial directions.

Well-being, Inclusivity & Campus Resources

Mental Health & Wellness Support

As graduate students, you face intense academic, personal, and career-related pressure. Your well-being is of paramount importance to the ECE Department.

Counseling and Wellness Center (CWC):

Provides counseling, mental health assistance, and support groups.

counseling.ufl.edu | (352) 392-1575

U Matter, We Care:

A UF-wide initiative offering aid and safety nets to students in distress.

umatter.ufl.edu | umatter@ufl.edu

Accommodations & Inclusion

Disability Resource Center (DRC):

Students requesting accommodations must register first with the DRC and submit accommodation letters to the instructor early.

drc.dso.ufl.edu | (352) 392-8565

Diverse & Inclusive Learning:

We strive to create a respectful, safe, and academically stimulating learning environment where all backgrounds and perspectives are valued.

Academic Integrity & Professional Ethics

Graduate-Level Expectation of Integrity

In this course, we treat you as junior researchers, professional engineers, and respected scholars. I operate under a strong starting assumption of mutual respect and high ethical standards. I do not expect or tolerate academic dishonesty.

Strict Collaboration Policies:

Independent Work: All submitted homework codes, calculations, and exam papers must represent your own individual effort.

Discussion vs. Duplication: Discussing high-level designs and debugging systems setups with classmates is encouraged, but sharing files, copying code, or duplicating written analysis is strictly forbidden.

Zero Tolerance: Violations will result in immediate failing grades ('E') and formal reporting to the Dean of Students Office.

What is "Computer Architecture"?

Defining the Discipline of Hardware Design

Historically, computer architecture was synonymously associated with Instruction Set Architecture (ISA) design—the process of determining registers and instructions. Today, the discipline encompasses a much broader multi-dimensional design space under physical limits.

> *"Computer architecture is the science and art of selecting and interconnecting hardware components to create computers that meet functional, performance, and cost goals."*
>
> **— Professor Gurindar Sohi, University of Wisconsin-Madison**

The Three-Level Taxonomy of Systems

Instruction Set Architecture

The Software Contract

The programmer-visible software-hardware boundary and state model.

- Defines instructions, opcode options, registers, memory model, addressing modes.
- Software compiled to an ISA runs on any processor executing that ISA.
- Examples: RISC-V, ARMv9, x86-64, MIPS.

Microarchitecture

Processor Organization

The high-level structural organization of the processor core.

- Determines pipelining depth, branch predictors, cache structures, execution ports.
- Completely hidden from the ISA; compilers generally do not need to see it.
- Different microarchitectures execute the same software at different speeds.

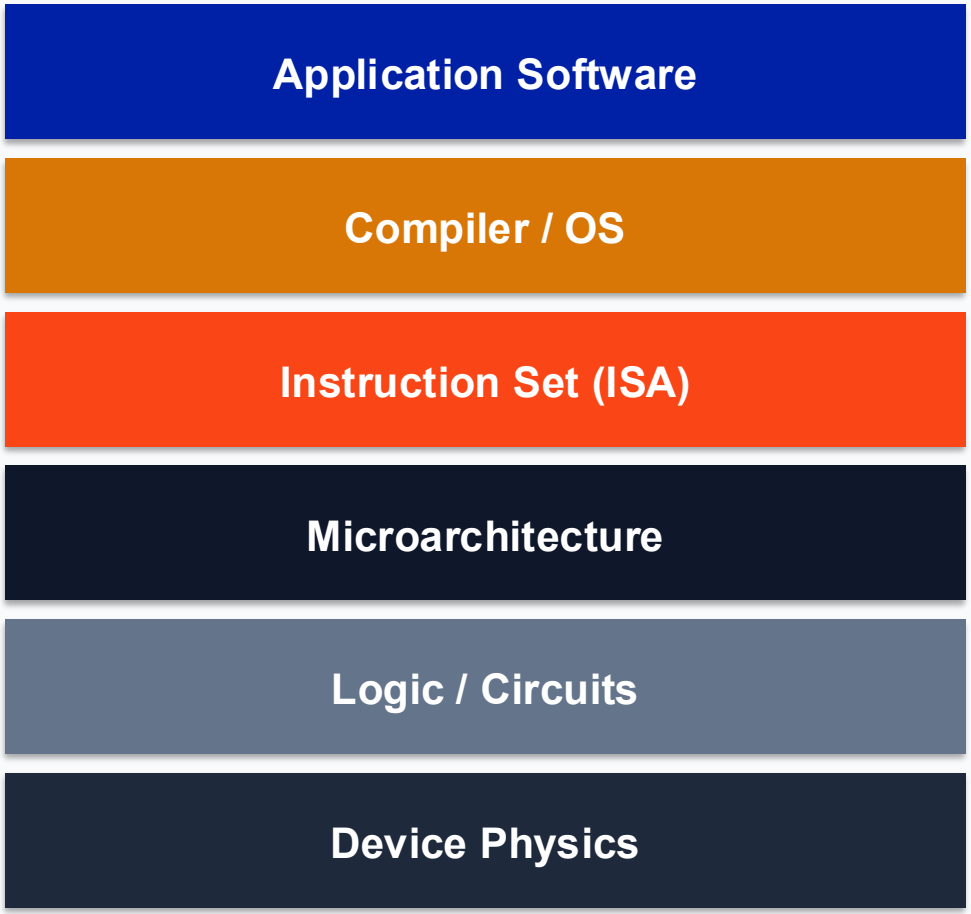
Physical Implementation

Transistor & Logic Design

The physical circuit layout, layout trees, and manufacturing physics.

- Defines transistor geometry, logic gates, physical copper wires, and clock networks.
- Determined by fabrication nodes (e.g., TSMC 3nm, Intel 18A, GaN technology).
- Affects operating frequency limits, power bounds, chip packaging, and manufacturing cost.

The Systems Abstraction Stack



Why Abstraction Stack Matters

Managing Complexity: Abstraction isolates programmers from hardware changes, enabling scalable software structures.

The Traditional Separation: Historically, abstraction boundaries allowed substantial independent development.

The Abstraction Tax: General-purpose abstraction models introduce execution overhead, power inefficiencies, and structural vulnerability.

Modern Necessity of Cross-Layer Co-design: Physical boundaries (power walls, security bugs like Meltdown) require looking across multiple layers concurrently to deliver performance, trust, and acceleration.

The Instruction Set Architecture (ISA)

ISA acts as the absolute interface contract between software development and physical execution units.

Core ISA Components

State Space: Registers (Integer, Float, Vector, Control), program counter, and memory organization.

Operation Formats: Instruction lengths, addressing modes, and registers configuration.

Instruction Classes: Arithmetic and logical (add, xor), data transfer (load, store), control flow (branch, jump).

Execution & Memory Contract

Memory Addressing Model: Byte-addressability vs. word alignment, virtual vs. physical schemes.

Exceptions & Interrupts: Interrupt types, recovery points, and architectural state rollback.

Memory Consistency Model: Rules for order in multi-core systems (e.g., sequentially consistent vs. relaxed).

Microarchitecture (Organization)

The physical organization of the hardware components that implements the ISA contract. It is not part of the functional ISA contract, but compilers absolutely may tune for particular microarchitectures.

Execution Optimization Mechanics

Pipelining Depth & Width: Determining the stages (IF, ID, EX, MEM, WB) and parallel execution slots.

Instruction Buffering: Instruction queues, Reservation Stations, and Reorder Buffers.

Dynamic Speculation: Branch predictors (BTB, direction predictors), speculative branch execution, and architectural state recovery.

Memory and Interconnection structures

Cache Hierarchy: Sizing, associativity, and replacement rules of L1 instruction/data, L2, and L3 caches.

Prefetching Algorithms: Stride, stream, and spatial prediction to fetch cache blocks early.

Internal Interconnects: Execution bypass networks, crossbars, rings, and shared memory networks.

Physical Implementation & Technology

The physical mapping of logical units to silicon structures.

Physical Parameters

Transistor Technology: FinFET, Gate-All-Around (GAA), and ultra-thin body structures.

Silicon Packaging: Interposers, high-density 2.5D/3D stacking, and chiplet designs.

Circuit Layout: Power delivery networks, clock trees, and layout floorplans.

The Physical Impact

Limits Operating Frequency: Maximum path delays define clock constraints.

Interconnect and Thermal Boundaries: Transistor/wire resistance and capacitance define power dissipation limits.

Economic Realities: Die yield rates directly shape system layouts and core sizes.

Course Roadmap: What You Know → What We Will Learn

Bridging undergraduate digital logic concepts with graduate-level quantitative computer systems design.

Previous Courses (Baseline Foundations)

Digital Logic & RTL: Logic gates, flip-flops, multiplexers, and hardware description.

Basic Datapaths: Constructing simple single-cycle register-to-register datapaths.

Basic ISA Concepts: Writing assembly code and understanding instruction formats.

Introductory Pipelining & Caches: Simplified in-order execution pipelines and standard direct-mapped/set-associative caches.

This Course (EEL 5764: Quantitative Focus)

Quantitative Design: Exact mathematical metrics, simulation modeling in gem5, cost/yield analysis.

Out-of-Order & Speculation: Dynamic scheduling (Tomasulo), branch prediction, reorder buffers.

Advanced Memory Systems: High-performance memory caches, DRAM, prefetching, and directory protocols.

Parallelism & Acceleration: Multi-core cache coherence, GPUs, SIMD vectors, and AI domain accelerators (systolic arrays).

Same ISA, Different Microarchitecture

The ISA is a functional interface contract. The microarchitecture defines the physical design choices.

Simple In-Order Core

Low-Complexity Implementation

Single-issue pipeline

No branch prediction tables

Executes 1 instruction/cycle max

Target: Microwatt microcontrollers

Focus: Lowest area and leakage

Superscalar OoO Core

Dynamic Instruction Flow

4-issue out-of-order execution

Advanced branch predictor

Executes up to 4 instructions/cycle

Target: Desktop and smartphone CPUs

Focus: High single-thread performance

High-Performance Multicore

Throughput Scalability

Many specialized in-order cores

Massive vector/tensor units

Massive memory cache sharing

Target: High-Performance Computing / AI

Focus: Extreme parallel bandwidth

The Architectural Motif: Four Core Questions

For every microarchitectural mechanism we analyze this semester, we will interrogate it using this structured 4-part framework:

1. What problem is solved?

Identification of the physical or software bottleneck. Is it branch hazard delay? Memory access latency? Vector alignment limits?

2. What mechanism is used?

The specific hardware, pipeline, or protocol innovation designed to mitigate or resolve the bottleneck (e.g., branch predictor, victim cache).

3. What does it cost?

The hardware tax: additional transistors, silicon die area, physical complexity, dynamic and static power leakage, and design verification time.

4. How is it evaluated?

Quantitative evaluation: How do we prove it is worthwhile? Cycle-level gem5 simulations, workload benchmarks, and mathematical formula models.

The Semester Journey: Walk Through the Stack

How we will systematically analyze the hardware stack over the next 15 weeks:

Weeks 1 - 3

Fundamentals

Performance metrics, RISC-V programming instruction set architecture, compilation models.

Weeks 4 - 7

The Processor

Instruction-Level Parallelism (ILP), pipelining, Tomasulo, branch prediction, Out-of-Order, speculation.

Weeks 8 - 11

Memory Hierarchy

SRAM caches, prefetching, virtual memory, physical DRAM, TLBs.

Weeks 12 - 15

Parallel Systems

Multi-core coherence, SIMD Vector processing, GPUs, Neural Processing accelerators (TPUs).

Multi-Dimensional Design Space & Parameters

Architects optimize designs specifically for targets. No single parameter rules every domain.

Traditional Design Dimensions

Performance: Latency (execution time) vs. throughput (workloads/sec).

Cost: Silicon fabrication, packaging, and testing costs.

Power & Energy: Peak power limits vs. total energy consumption bounds.

Complexity: Design compilation, testing times, and layout constraints.

Advanced & Emerging Dimensions

Security & Trust: Hardware side-channel protection (e.g., Meltdown/Spectre mitigation).

Robustness & Reliability: Resilience to soft errors (cosmic radiation) and aging.

Testability: Built-in self-test (BIST) capabilities and scan chains.

Silicon Economics: Conceptual Cost of ICs

The economics of semiconductor manufacturing are highly non-linear. The cost of a finished microchip is driven by defect distributions on physical wafers.

Economic Drivers of Silicon Cost

Wafer Costs: Slicing silicon ingots is a fixed, highly expensive process.

Final Test Yield: Not every die works. Lower yield dramatically amplifies the unit cost of functional chips.

Packaging tax: Sophisticated packaging (like TSMC CoWoS 3D chiplets) contributes heavily to modern high-performance SoC costs.

Foundational Cost Formulations

Cost of IC = (Cost of Die + Cost of Testing + Cost of Packaging) / Final Test Yield

💡 Takeaway: If final test yield is low, the cost of the finished IC increases exponentially. Maximizing yield is a core design constraint.

Estimating Dies Per Wafer & Yield

The relationship between physical die size and overall manufacturing costs is inherently non-linear due to defect models.

Mathematical Formulations

Dies Per Wafer ≈ [π * (Wafer Radius)² / Die Area] - [π * Wafer Diameter / √(2 * Die Area)]

Die Yield = Wafer Yield * (1 + Defects Per Unit Area * Die Area / N)^-N

Defects Per Unit Area: Typical physical impurities (0.4 to 1.0 per cm²).

N: Complexity parameter (commonly 11.5 to 13.5 for modern processes).

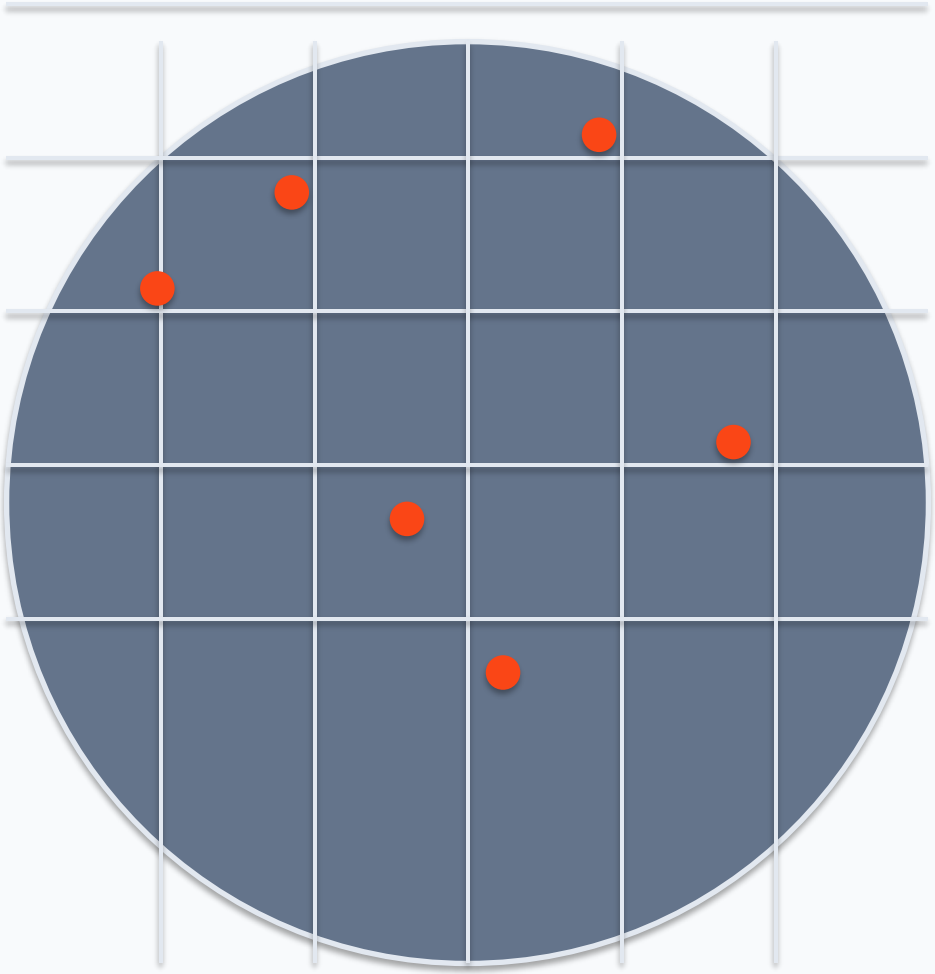
The Exponential Cost Trap

Non-linear scaling: Doubling the die area decreases yield exponentially, not linearly, because a larger die is much more likely to contain a defect.

Edge Loss subtraction: The subtraction term in the Dies Per Wafer formula accounts for the non-square dies around the wafer perimeter.

Chiplet motivation: Rather than manufacturing one giant monolithic die, modern systems (e.g., AMD Ryzen) split the design into smaller chiplets to maximize silicon yield and reduce costs.

Visualizing Defect Density & Die Yield



Yield Impact of Die Sizes

Defects (Orange Spots) are randomly distributed on the physical silicon wafer during chemical processing.

If die size is small: A defect only kills one small block of logic. The remaining adjacent dies remain functional (yielding high overall percentage).

If die size is extremely large: A single defect kills the entire large monolithic processor. The total yield plunges dramatically.

This economic reality forces a physical size ceiling for general-purpose processors, capping die sizes around 600-800 mm^2.